MPI Lab Question Bank

1. ALP to convert from ºC to ºF

- Store the temperature value in **°C** in the data segment.
- Use the variable name **Deg_Cent** for this location in the assembly program
- Reserve a location in the data segment to store the temperature in **°F**.
- Use the variable name **Deg_Far** for this location in the assembly program
- Implement the **°C to °F** conversion formula using arithmetic instructions.
- Store the final result in **Deg_Far**.

2. ALP to convert from ºF to K

- Store the given temperature in °F in the data segment (memory).
- Use the variable name Deg_Far for this location in the assembly program
- Reserve a location in the data segment to store the temperature in K.
- Use the variable name Deg_Kel for this location in the assembly program
- Use arithmetic instructions to implement the °F to K conversion formula.
- Store the final answer in Deg_Kel.

3. ALP to find factorial of a number

- Prompt the user to enter a number between 1 and 8 (you can show a message "Enter the number between 1 and 8")
- Once the user enters the number you can compute the factorial of the number and store it in data segment (memory) which is mapped to a variable 'FACT'

4. ALP to find factorial of a number using recursion

- Prompt the user to enter a number between 1 and 8 (you can show a message "Enter the number between 1 and 8")
- Once the user enters the number you can compute the factorial of the number using recursion and store it in a memory location in the data segment which is mapped to a variable 'FACT'

5. ALP to find sum of first n natural numbers

- Prompt the user to enter a number between 1 and 20 (you can show a message "Enter the number between 1 and 20")
- Once the user enters the number you can compute the sum of the first n natural numbers and store it in data segment which is mapped to a variable 'NSUM'

6. **ALP to find the first N terms of the fibonacci series**
   - Store a byte value in the memory location defined by variable "N"
   - Create an empty array of N bytes in the memory, with array name as "FIB_Out"
   - Generate N terms of Fibonacci series and store them in array location "FIB_Out"

7. **Write an assembly program to find the Sum of Squares of First *n* Natural Numbers**
   - Read the value of number n ($1 \leq n \leq 57$) from a memory location DS:2600H using SI register and store it in AX register
   - Implement the following formula to calculate the sum of n natural numbers with the help of arithmetic instructions (MUL, DIV, ADD)

$$SUM = 1^2 + 2^2 + \cdots n^2 = \frac{n(n+1)(2n+1)}{6}$$

   - Store the final sum value in the data segment at the memory location DS:2800H using DI register.

8. **Write an assembly program to compute an 8-bit checksum for a byte stored in memory:**
   - Read an 8-bit input value x from the memory location DS:6000H.
   - Compute the checksum using the following function:

$$Checksum = \left(x + (x \ll 3)\right) XOR\ (x \gg 1)$$

   - Store the computed checksum at memory location DS:6001H.

   Note: `<< n` denotes a left shift by *n* bit positions and `>> n` denotes a right shift by *n* bit positions.

9. **ALP to sort the elements of an array in ascending order using bubble sort**
   - Store a byte value representing the number of elements in an array at a memory location defined by variable "N"
   - Store array of "N" bytes in the memory, with array name as "ARRAY"
   - Sort the elements of array "ARRAY" in ascending order using Bubble sort algorithms

10. **ALP to sort the elements of an array in descending order using bubble sort**
   - Store a byte value representing the number of elements in an array at a memory location defined by variable "N"
   - Store array of "N" bytes in the memory, with array name as "ARRAY"
   - Sort the elements of array "ARRAY" in descending order using Bubble sort algorithms

11. ALP to find the GCD of two numbers

- Store a byte value in the memory location defined by variable "Num1"
- Store a byte value in the memory location defined by variable "Num2"
- Create an empty memory location defined by variable "GCD_Out"
- Find GCD of two numbers in "Num1" and "Num2" and store the result in the memory location defined by GCD_Out

12. ALP to find the LCM of two numbers

- Store a byte value in the memory location defined by variable "Num1"
- Store a byte value in the memory location defined by variable "Num2"
- Create an empty memory location defined by variable "LCM_Out"
- Find LCM of two numbers in "Num1" and "Num2" and store the result in the memory location defined by LCM_Out

13. Write an assembly program to perform Matrix Addition of two 2×2 matrices

- Store the elements of a **2×2 matrix A** (4 bytes) in memory with the label **"ARRAY1"**
- Store the elements of a **2×2 matrix B** (4 bytes) in memory with the label **"ARRAY2"**
- Add the corresponding elements of **ARRAY1** and **ARRAY2** and store the **resultant 2×2 matrix** in memory starting at the destination location (**DS:7000H**)

14. Write an assembly language program to **convert 8-bit binary data to BCD**

- Store the 8-bit binary data to be converted into BCD in the data segment and use the variable name Bin_Data for this location in the assembly program.
- Reserve appropriate locations in the data segment to store the BCD result and use the variable names BCD_Res (for tens and units) and BCD_Hund (for hundreds) in the assembly program.
- Access the data segment and load the value stored in Bin_Data.
- Convert the given 8-bit binary data into BCD format using assembly language instructions.
- Store the final BCD result in BCD_Res and BCD_Hund.

15. ALP to copy a character string from one location to another and display both

- Store the string length in the memory location defined by variable name "StrLen"
- Store a character string in the memory location defined by variable "STR"
- Reserve memory defined by variable "STRCPY" for storing the copied string
- Copy the string from location "STR" to "STRCPY" and display both the strings on the console in two different lines.

16. ALP to reverse a string, and print both the original and new versions of the string

- Store the string length in the memory location defined by variable name "StrLen"

- Store a character string in the memory location defined by variable “STR”
- Reserve memory defined by variable “REVSTR” for storing the reversed string
- Reverse the string at location “STR” and store it at “REVSTR”.
- Display both the strings on the console in two different lines.

17. ALP to check if a given string is a palindrome

- Store the string length in the memory location defined by variable name “StrLen”
- Store a character string in the memory location defined by variable “STR”
- Check if the given string is a palindrome, and if it is, display a message accordingly in the console, showing the original and reversed strings to be equal. If it is not the same, display a message indicating the same and display both original and reversed strings to indicate the difference.

18. ALP to reverse a character string without using any additional memory locations, and display both original and reversed strings

- Store the string length in the memory location defined by variable name “StrLen”
- Store a character string in the memory location defined by variable “STR”
- Display the original string.
- Reverse the string at location “STR” and store it at the same location.
- Display the reversed string.

19. Write an ALP to compare two strings of 20 bytes

- Store a string of 20 bytes in memory with the name **STR1**.
- Store another string of 20 bytes in memory with the name **STR2**.
- Compare the two strings byte by byte.
- If the strings are equal, store **1** in register **BL**; otherwise, store **0** in **BL**.

20. ALP to multiply two 8-bit numbers without using MUL instruction

- Store two 8-bit numbers in memory locations defined by variables **NUM1** and **NUM2**.
- Reserve a memory location to store the result and use the variable name **PROD**.
- Access the data segment and load the values of **NUM1** and **NUM2**.
- Perform multiplication using **shift and addition operations**.
- Store the final result in **PROD**.

21. Write an ALP code to find the square root of a perfect square

- Store the given number (perfect square) in a memory location defined by variable **NUM**.
- Reserve a memory location to store the square root and use the variable name **ROOT**.
- Access the data segment and load the value stored in **NUM**.
- Implement the repeated subtraction method using assembly language instructions.
- Store the final square root in **ROOT**.

22. **Write a program in 8086 ALP to add data in one block of memory with data in another block of memory (Array Addition)**

    Assume,

    No of elements in the block: 5

    Block 1: 23h, 42h, 63h, 77h, 25h

    Block 2: 31h, 12h, 50h, 33h, 20h

    After the program Block 2 should be as below:

    Block 2: 54h, 54h, B3h, AAh, 45h

23. **ALP to convert a BCD number to binary using a procedure**
    - Store a BCD number in a memory location defined by variable **BCDNum**.
    - Load the BCD number into register **AL**.
    - Call a procedure named **BCDtoBin** to convert the BCD number into binary format.
    - Perform the conversion within the procedure **BCDtoBin**.
    - Store the resulting binary value in register **AL**.

24. **ALP to convert 8-bit binary data to Gray code**
    - Store the 8-bit binary data in a memory location defined by variable **BIN_Data**.
    - Reserve a memory location to store the Gray code result and use the variable name **GRAY_Out**.
    - Access the data segment and load the value stored in **BIN_Data**.
    - Convert the given 8-bit binary data into **Gray code** using assembly language instructions.
    - Store the final Gray code result in **GRAY_Out**.

25. **ALP to search for an element in an array**
    - Store the number of elements in the array in a memory location defined by variable **N**.
    - Store an array of **N** bytes in memory with the name **ARRAY**.
    - Store the element to be searched in a memory location defined by variable **KEY**.
    - Search for the element in the array.
    - Store the result in a memory location defined by variable **RESULT**.

26. ALP to count the **number of 1's** in an 8-bit number
    - Store an 8-bit number in a memory location defined by variable **NUM**.
    - Count the number of 1's in the given number.
    - Store the result in a memory location defined by variable **COUNT**.

27. **ALP to check if the number of odd and even elements in an array are equal**
    - Store the number of elements in the array in a memory location defined by variable **N**.
    - Store an array of **N** bytes in memory with the name **ARRAY**.
    - Count the number of odd and even elements in the array.
    - If the number of odd and even elements are equal, store **1H** in a memory location defined by variable **RESULT**; otherwise, store **0H** in **RESULT**.

28. ALP to find the Hamming distance between two numbers

- Store two numbers in memory locations defined by variables **NUM1** and **NUM2**.
- The Hamming distance is defined as the number of bit positions at which the corresponding bits differ (For example, the Hamming distance between 0100 and 1100 is 1, and between 0101 and 1011 is 3)
- Compute the Hamming distance between **NUM1** and **NUM2**.
- Store the result in a memory location defined by variable **DIST**.

29. ALP to **generate multiplication table** of a number

- Store the given number in a memory location defined by variable **NUM**.
- Generate the multiplication table for this number.
- Store the resulting values in an array defined by variable **TABLE**.

30. ALP to find the roots of a quadratic equation (Δ is a perfect square)

- Store the coefficients **a**, **b**, and **c** in memory locations defined by variables **A**, **B**, and **C**.
- Reserve memory locations to store the two roots and use variable names **ROOT1** and **ROOT2**.
- Compute the value of the discriminant $\boldsymbol{\Delta = b^2 - 4ac}$.
- Compute the square root of **Δ** using the **repeated subtraction method**.
- Use the formula to calculate the two roots.

$$x = \frac{-b \pm \sqrt{\Delta}}{2a}$$

- Store the calculated roots in **ROOT1** and **ROOT2**.